



Affest

PROJECT DECK / WHITEPAPER

Affest

Verified cross-chain automation for self-driving portfolios.

THE ONE-LINE IDEA

Create and control an automated portfolio from a dashboard. Manage it from any AI agent through MCP, while Creditcoin enforces the user's boundaries.

BUIDL CTC 2026 Fall / Testnet submission material

01 The problem

AI agents can read more systems than people can monitor, but financial automation has a trust problem. A backend can say that an event happened. A model can misread a message. A compromised relayer can try to overreach. None of those claims should become a withdrawal permission.

Affest separates observation from authority. Agents may read, explain, plan, and request. The chain decides whether a verified source fact satisfies a user-approved policy.

The user story

A HUMAN RULE

Keep 70% of my portfolio in stable assets and 30% in risk assets. When I receive at least 1,000 USDC on Ethereum, prepare a rebalance. Automatically execute it below my \$500 limit; otherwise ask me.

02 The product

SURFACE	WHAT IT DOES	USER VALUE
Investor dashboard	Create vaults, inspect allocations, review deterministic strategy rules, approve actions, and pause access.	Human control stays visible.
Remote MCP	Expose read, planning, proof, and action tools through a model-independent Streamable HTTP server.	Use Claude, GPT, or another MCP client.
Proof worker	Monitor Sepolia, wait for Attestcoin, build proofs, simulate, and coordinate Creditcoin submission.	Automation can run without trusting the worker.
Creditcoin contracts	Enforce assets, adapters, limits, expiry, replay protection, pause state, and execution mode.	Policy is on-chain, not a database promise.

03 How the trust model works

Affest uses a simple rule: data may be observed off-chain, but permission must be decided on-chain.

01 Observe worker sees a source log	02 Attest Attestcoin proves inclusion	03 Decode receipt and event are checked	04 Authorize strategy limits are checked	05 Act vault executes an allowlisted action
--	--	--	---	--

Three execution modes

MODE	BEHAVIOUR
Approval	The agent prepares an unsigned action. The user reviews and signs it.
Automatic	A restricted executor acts only when the on-chain policy allows the exact proof-backed request.
Hybrid	Small actions execute automatically. Larger actions become approval requests. This is Affest's recommended mode.

04 Attestcoin integration

Affest uses Attestcoin as a proof layer for Ethereum activity. It does not treat a log index, database row, or AI response as proof.

SOURCE

Ethereum Sepolia hosts PortfolioSignalEmitter. Its emitSignal function emits a unique PortfolioSignal event with signalId, user, asset, amount, and signalType.

PROOF

The worker waits for the source block to reach the Attestcoin attested height, then uses the official USC SDK to build the proof from the transaction hash.

VERIFICATION

On Creditcoin, AffestAttestationVerifier invokes the official native query verifier and decoder. It requires successful receipt status, the right source contract, exact event signature, matching fields, and an unused event key.

ACTION

Only after verification does AffestExecutor call the strategy manager and vault. The adapter and assets are allowlisted, and the request is bounded by the user's policy.

Why Attestcoin is necessary

Without Attestcoin, Affest would be trusting its own worker to report what happened on Ethereum. With Attestcoin, the worker supplies evidence and Creditcoin verifies the evidence before the policy can run. The worker coordinates; the contract judges.

05 MCP for agent-first finance

The MCP server is model-independent. Any compatible client can use the same typed tools, credentials, scopes, and audit trail.

LAYER	EXAMPLES	GUARDRAIL
Read	get_portfolios, get_strategy, get_pending_actions	No signing. No secret material.
Plan	draft_strategy, validate_strategy, preview_rebalance	Deterministic schema before permission.
Proof	check_attestation_readiness, build_attestcoin_proof	Evidence is explicit and inspectable.
Action	request_rebalance, pause_strategy, revoke_agent_access	Preview-first, scoped, idempotent, audited.

06 Security by design

Affest assumes the agent, worker, database, and relayer can fail. The design limits what each layer can do.

Threat	Boundary
Malicious agent	Scoped bearer credential, preview-first actions, no arbitrary calldata, instant revoke.
Compromised relayer	Cannot withdraw arbitrarily. Contracts enforce allowlists, limits, expiry, and pause state.
Tampered or replayed proof	Official verifier path, receipt/event checks, and unique source event keys.
Database compromise	Database coordinates UX only. On-chain state remains the permission source.
Prompt injection	Natural language becomes a visible deterministic draft; it never becomes arbitrary execution rights.

07 MVP status and demo

The MVP is deliberately small: one vault per user, two portfolio assets, one Sepolia signal type, one deterministic strategy shape, three execution modes, a remote MCP server, and a real Attestcoin verification boundary.

The judge journey

01 Connect Creditcoin wallet	02 Create strategy + limits	03 Attach MCP credential	04 Signal Sepolia event	05 Verify Attestcoin on CC3
--	---	--	---	---

WHAT IS REAL

The dashboard, MCP server, Creditcoin contracts, source event contract, proof schema boundary, replay rules, and verifier integration are implemented. The worker is persistent and restart-safe.

WHAT IS LABELLED

CC3 exchange execution uses a fixed demo adapter because a reliable documented CC3 DEX route and liquidity guarantee were not confirmed. This boundary is shown in the product and documentation.

WHAT TO REMEMBER

An AI portfolio manager can react to activity on another blockchain without trusting the backend and without receiving unrestricted access to the user's funds.

Links

Dashboard: affest.cefo.dev

MCP endpoint: mcp.affest.cefo.dev/mcp

Source contract: [Sepolia explorer](#)

Testnet only. Never use production funds or credentials in this demo.